



INFSCI 2591

ALGORITHM DESIGN

Instructor: Pengfei Zhou

RM 710, IS Building

pez60@pitt.edu



Video games

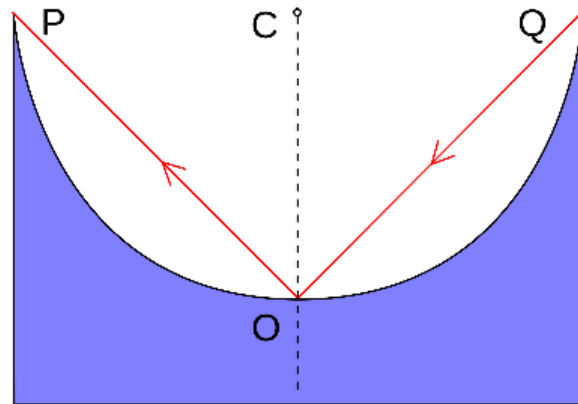
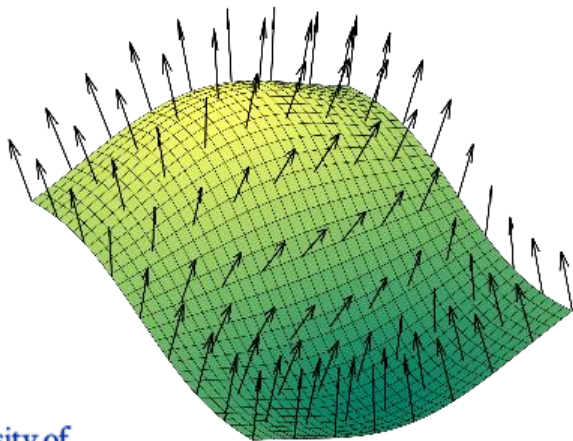


Physics and Graphics



Calculating Inverse square root

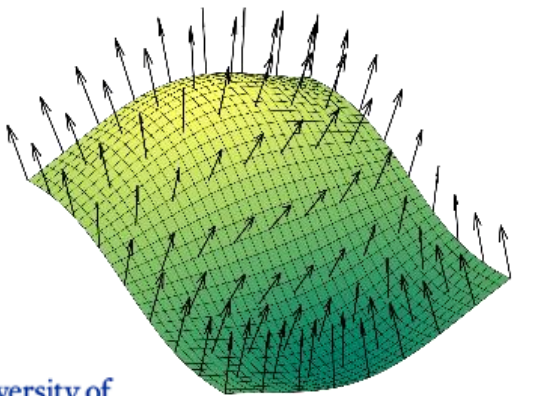
- Applications: Physics and Engineering, Computer Graphics, etc.



Calculating Inverse square root

- Vector normalization

(x, y, z) $\xrightarrow{\text{normalization}}$



$$\left(\frac{x}{\sqrt{x^2 + y^2 + z^2}}, \frac{y}{\sqrt{x^2 + y^2 + z^2}}, \frac{z}{\sqrt{x^2 + y^2 + z^2}} \right) \\ = \frac{1}{\sqrt{x^2 + y^2 + z^2}} (x, y, z)$$

Multiplication and addition are fast, so the problem left is to calculate inverse square root.

$$f(x) = \frac{1}{\sqrt{x}}$$

Solution 1

```
float y = 1 / sqrt(x);
```

Problem: Very slow.

```
#include <math.h>

double sqrt(double x) {
    if (x == 0) {
        return 0;
    }

    double guess = x / 2.0; // Initial guess
    double tolerance = 1e-10; // Desired precision

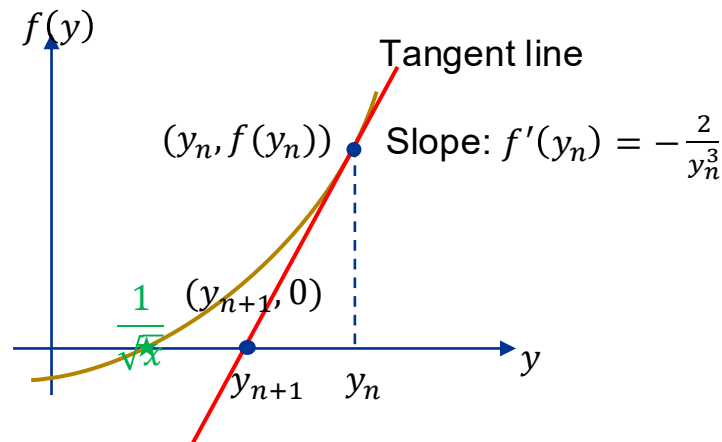
    while (fabs(guess * guess - x) > tolerance) {
        guess = (guess + x / guess) / 2.0;
    }

    return guess;
}
```

Solution 2: Newton's method

- Approximate $\frac{1}{\sqrt{x}}$ using $f(y) = \frac{1}{y^2} - x$

$$\begin{aligned} y_{n+1} &= y_n - \frac{f(y_n)}{f'(y_n)} \\ &= y_n \left(\frac{3}{2} - \frac{1}{2} x y_n^2 \right) \end{aligned}$$



Solution 3



- Source code from Quake III Arena, a first-person shooter video game

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

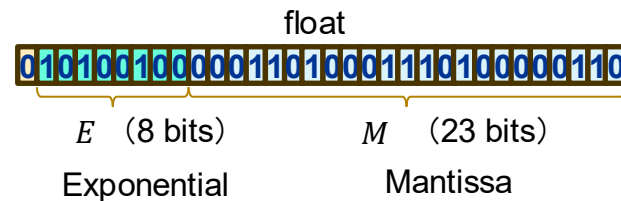
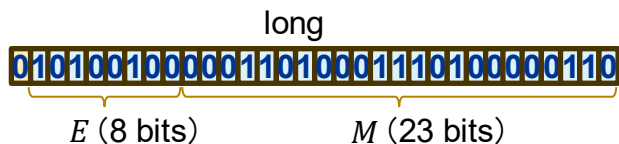
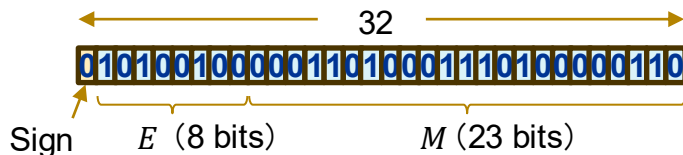
    x2 = number * 0.5F;
    y = number;
    i = * ( long * ) &y;                // evil floating point bit level hacking
    i = 0x5f3759df - ( i >> 1 );        // what the fuck?
    y = * ( float * ) &i;
    y = y * ( threehalfs - ( x2 * y * y ) ); // 1st iteration
    // y = y * ( threehalfs - ( x2 * y * y ) ); // 2nd iteration, this can be removed

    return y;
}
```


Solution 3

- How is the number stored in computers?

```
float Q_rsqrt( float number )  
{  
    long i;  
    float x2, y;  
    const float threehalfs = 1.5F;  
  
    x2 = number * 0.5F;  
    y = number;
```



$$2^{23} * E + M$$

$$\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127}$$



Solution 3

- Floating-point representation
 - Scientific notation
 - IEEE 754

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y  = number;
```

Exponential Mantissa

00111111000100000000000000000000

E

M

M	0100111001000000000000
$2^{23} \cdot E$	10001001 00000000000000000000
$2^{23} \cdot E + M$	10001001 01001110010000000000

Bit representation: $2^{23} * E + M$

Actual number: $\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127}$

Solution 3

- Approximation using logarithmic operations

Actual number

$$\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127}$$

↓ Apply logarithmic operation

$$\begin{aligned} & \log_2 \left(\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127} \right) \\ &= \log_2 \left(1 + \frac{M}{2^{23}}\right) + \log_2 2^{E-127} \\ &= \log_2 \left(1 + \frac{M}{2^{23}}\right) + E - 127 \end{aligned}$$



Solution 3

- Approximation using logarithmic operations

Actual number

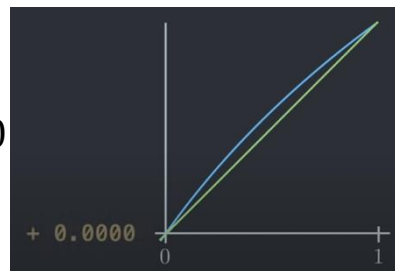
$$\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127}$$

↓ Apply logarithmic operation

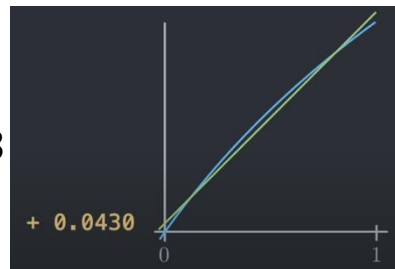
$$\begin{aligned} & \log_2 \left(\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127} \right) \\ &= \log_2 \left(1 + \frac{M}{2^{23}}\right) + \log_2 2^{E-127} \\ &= \log_2 \left(1 + \frac{M}{2^{23}}\right) + E - 127 \\ &\approx \frac{M}{2^{23}} + \mu + E - 127 \end{aligned}$$

$$\log_2(1+x) \approx x + \mu, \quad \mu \in [0, 1)$$

$$\mu = 0$$



$$\mu = 0.043$$



Solution 3

- Approximation using logarithmic operations

$$\begin{aligned} & \frac{M}{2^{23}} + \mu + E - 127 \\ &= \frac{M}{2^{23}} + E + \mu - 127 \\ &= \frac{1}{2^{23}} (M + E * 2^{23}) + \mu - 127 \end{aligned}$$

Bit representation

Apply logarithmic
operation

Actual number

$$\left(1 + \frac{M}{2^{23}}\right) * 2^{E-127}$$



Solution 3

- Bit manticulation

```
i = * ( long * ) &y;           // evil floating point bit level hacking
i = 0x5f3759df - ( i >> 1 );   // what the fuck?
y = * ( float * ) &i;
```

float y

- IEEE 754
- no bit manipulation

long i

- ordinary 32-bit number
- has bit manipulation

110 = 6	111 = 7
1100 = 12	1110 = 14
11 = 3	11 = 3

i = * (long *) &y;

0x7F5423A6

0x7F5423A6

...
0 1011 100001010
...



Solution 3

0x5F3759DF = 01011111001101110101100111011111

```
i = * ( long * ) &y; // evil floating point bit level hacking
i = 0x5f3759df - ( i >> 1 ); // what the fuck?
y = * ( float * ) &i;
```

i is a long integer, y is a float number. They have the same bits stored in the computer

$$y = \frac{1}{\sqrt{x}} \Rightarrow \log_2(y) = \log_2\left(\frac{1}{\sqrt{x}}\right) = \log_2\left(x^{-\frac{1}{2}}\right) = -\frac{1}{2}\log_2(x)$$

Replace $\log_2(y)$ and $\log_2(x)$ using bit presentation $\log_2(y) \approx \frac{1}{2^{23}}(M_y + E_y * 2^{23}) + \mu - 127$ and $\log_2(x) \approx \frac{1}{2^{23}}(M_x + E_x * 2^{23}) + \mu - 127$, we have

$$\Rightarrow \frac{1}{2^{23}}(M_y + E_y * 2^{23}) + \mu - 127 = -\frac{1}{2}\left(\frac{1}{2^{23}}(M_x + E_x * 2^{23}) + \mu - 127\right)$$

$$M_y + E_y * 2^{23} = \frac{3}{2} * 2^{23} (127 - \mu) - \frac{1}{2}(M_x + E_x * 2^{23})$$



y_{long}

$$= \frac{3}{2} * 2^{23} (127 - \mu) - \frac{1}{2} * x_{long}$$



$$= 0x5f3759df - (x_{long} >> 1)$$



University of
Pittsburgh



Solution 3: Fast InvSqrt()

- Have a good initiate of the approximation

```
float Q_rsqrt( float number )
{
    long i;
    float x2, y;
    const float threehalfs = 1.5F;

    x2 = number * 0.5F;
    y = number;
    i = * ( long * ) &y;           // evil floating point bit level hacking
    i = 0x5f3759df - ( i >> 1 );  // what the fuck?
    y = * ( float * ) &i;

    y = y * ( threehalfs - ( x2 * y * y ) ); // 1st iteration
    // y = y * ( threehalfs - ( x2 * y * y ) ); // 2nd iteration, this can be removed

    return y;
}
```



Solution 3: Fast InvSqrt()

- Recipe

The algorithm computes $\frac{1}{\sqrt{x}}$ by performing the following steps:

1. Alias the argument x to an integer as a way to compute an approximation of the [binary logarithm](#) $\log_2(x)$
2. Use this approximation to compute an approximation of
$$\log_2\left(\frac{1}{\sqrt{x}}\right) = -\frac{1}{2}\log_2(x)$$
3. Alias back to a float, as a way to compute an approximation of the base-2 exponential
4. Refine the approximation using a single iteration of Newton's method.



Takeaways

- Bit manipulation in computers
- Using logarithmic operations to convert exponential expressions into multiplicative forms
- Newtown's method (iterations) for approximation
- Fast inverse square root algorithm